

# 编译器导论

## Chapter 1: Introduction to Compilers

基于南京大学计算机学院课程讲义

时清凯 | qingkaishi@nju.edu.cn

2026 年 6 月 10 日

# 目录

|                                         |           |
|-----------------------------------------|-----------|
| 前言                                      | 3         |
| <b>1 第一部分：什么是编译器？</b>                   | <b>4</b>  |
| 1.1 编译器的基本定义                            | 4         |
| 1.2 编译器 vs. 解释器 vs. 链接器                 | 4         |
| 1.2.1 编译器与解释器的区别                        | 4         |
| 1.2.2 编译器与链接器的关系                        | 5         |
| 1.2.3 链接的关键作用                           | 6         |
| 1.3 编译器的工作流程：三阶段架构                      | 6         |
| 1.3.1 前端（Front End）                     | 6         |
| 1.3.2 中端（Middle End）                    | 7         |
| 1.3.3 后端（Back End）                      | 7         |
| <b>2 编译流程详解</b>                         | <b>8</b>  |
| 2.1 词法分析（Lexical Analysis）              | 8         |
| 2.2 语法分析（Syntax Analysis）               | 9         |
| 2.3 语义分析（Semantic Analysis）             | 9         |
| 2.4 IR 生成（Intermediate Code Generation） | 10        |
| <b>3 中端优化</b>                           | <b>12</b> |
| 3.1 机器无关优化的特点                           | 12        |
| 3.2 优化示例                                | 12        |
| <b>4 后端：从 IR 到目标代码</b>                  | <b>13</b> |
| 4.1 指令选择（Instruction Selection）         | 13        |
| 4.2 寄存器分配（Register Allocation）          | 13        |
| 4.3 指令调度（Instruction Scheduling）        | 14        |
| <b>5 课程结构</b>                           | <b>15</b> |
| 5.1 内容划分                                | 15        |
| 5.2 实验安排                                | 15        |
| 5.3 课程信息                                | 15        |
| <b>6 第二部分：编译器开发工具</b>                   | <b>16</b> |
| 6.1 工具概览                                | 16        |
| 6.2 主要工具分类                              | 16        |

|                                              |           |
|----------------------------------------------|-----------|
| <b>7 第三部分：编译器的应用</b>                         | <b>17</b> |
| 7.1 高级编程语言的实现 . . . . .                      | 17        |
| 7.2 计算机体系结构的设计与优化 . . . . .                  | 17        |
| 7.3 广泛的应用领域 . . . . .                        | 17        |
| 7.4 程序翻译 (Transpilation) . . . . .           | 17        |
| 7.5 软件生产力工具 . . . . .                        | 17        |
| 7.6 安全保证工具 . . . . .                         | 18        |
| <b>8 第四部分：编程语言术语回顾</b>                       | <b>19</b> |
| 8.1 声明式语言 vs. 命令式语言 . . . . .                | 19        |
| 8.2 函数式编程 (Functional Programming) . . . . . | 19        |
| 8.3 逻辑编程 (Logic Programming) . . . . .       | 20        |
| 8.4 类型系统 (Type Systems) . . . . .            | 21        |
| 8.4.1 静态类型 vs. 动态类型 . . . . .                | 21        |
| 8.4.2 强类型 vs. 弱类型 . . . . .                  | 21        |
| 8.5 作用域 (Scoping) . . . . .                  | 22        |
| 8.6 函数调用 (Function Invocation) . . . . .     | 22        |
| 8.7 虚函数 (Virtual Functions) . . . . .        | 23        |
| <b>9 课程总结</b>                                | <b>24</b> |

## 前言

本文是对《编译器导论》课程第一章 (Introduction to Compilers) 的详细讲解笔记。原始讲义由南京大学计算机学院时清凯老师制作，共包含 131 页幻灯片。本文将内容重新组织为四个主要部分，并以适合阅读和学习的形式呈现：

1. **第一部分：什么是编译器？**—编译器的基本概念、编译器与解释器/链接器的区别、编译器的整体架构（前端、中端、后端）
2. **第二部分：开发工具**—编译器开发中使用的各类工具（词法/语法分析器生成器等）
3. **第三部分：编译器的应用**—编译器技术在不同领域的应用场景
4. **第四部分：编程语言术语回顾**—编程范型、类型系统、作用域与函数调用等基础概念

本文适合作为学习编译器原理的入门材料，建议读者配合原始讲义 Slides 一起使用。

# 1 第一部分：什么是编译器？

## 1.1 编译器的基本定义

核心概念

编译器（Compiler）是一种将源语言（Source Language）编写的程序翻译为目标语言（Target Language）程序的软件工具。

典型的编译场景包括：

- C/C++/Java → x86 机器码 / Java Bytecode
- 近年来还出现了 语言到语言的转译器（Transpiler），例如 C → Rust 的翻译工具

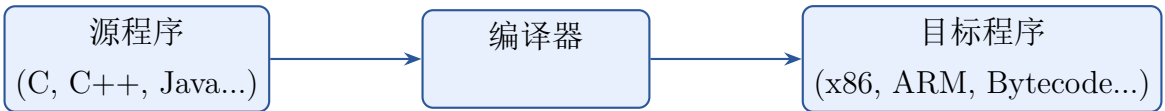


图 1: 编译器：源语言 → 目标语言

## 1.2 编译器 vs. 解释器 vs. 链接器

### 1.2.1 编译器与解释器的区别

核心概念

解释器（Interpreter）不将源程序翻译为另一种语言，而是直接解释并执行源语言中的语句。

| 特性   | 编译器               | 解释器                       |
|------|-------------------|---------------------------|
| 工作方式 | 先将整个程序翻译为目标代码，再执行 | 逐条读取源程序语句，直接执行            |
| 输出产物 | 生成独立的目标程序文件       | 不产生独立的可执行文件               |
| 执行效率 | 高（一次翻译，多次执行）      | 低（每次执行都需要解析）              |
| 典型语言 | C, C++, Rust, Go  | Python, Shell, JavaScript |

表 1: 编译器与解释器的对比

### 笔记

Java 是一个特殊案例：Java 使用了 **编译器 + 解释器 + JIT (Just-In-Time) 编译器** 的组合策略：

1. 首先由 `javac` 编译器将 Java 源码编译为字节码 (Bytecode)
2. 运行时由 JVM 的解释器逐条解释字节码
3. JIT 编译器将热点代码 (Hotspot) 编译为本地机器码以提高性能

## 1.2.2 编译器与链接器的关系

### 核心概念

**链接器 (Linker)** 将多个编译单元 (Compilation Units) 链接为单个文件，可以是可执行文件 (Executable)、静态库 (Static Library) 或动态库 (Dynamic Library)。

以 GCC 为例说明编译与链接的完整流程：

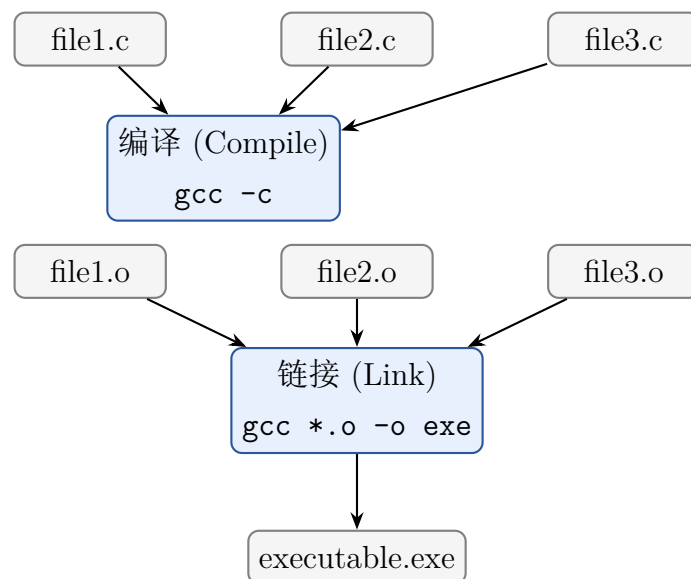


图 2: GCC 的编译与链接流程

### 1.2.3 链接的关键作用

#### 笔记

链接器的核心作用是符号解析 (*Symbol Resolution*)。例如：

- `file1.c` 中 声明并 调用了函数 `foo()`
- `file2.c` 中 定义 (实现) 了函数 `foo()`

编译阶段, `file1.o` 中会记录一个对 `foo` 的未解析引用；在链接阶段, 链接器会将其与 `file2.o` 中 `foo` 的定义绑定 (*bind*) 起来。

此外, 链接器还能链接第三方库 (`-lxxx` 选项)。

## 1.3 编译器的工作流程：三阶段架构

编译器通常分为前端 (*Front End*)、中端 (*Middle End*) 和后端 (*Back End*) 三个阶段。

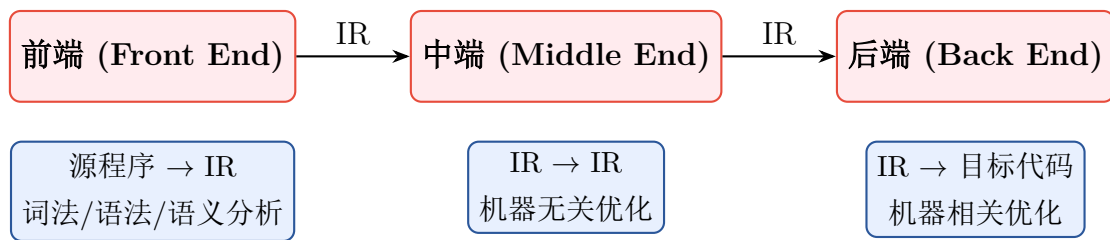


图 3: 编译器的三阶段架构

#### 1.3.1 前端 (Front End)

前端负责读取源文件中的文本, 并将其翻译为中间表示 (*Intermediate Representation, IR*)。主要步骤包括:

1. 词法分析 (*Lexical Analysis*): 将字符流转换为 Token 流
2. 语法分析 (*Syntax Analysis*): 根据 Token 流构建抽象语法树 (AST)
3. 语义分析 (*Semantic Analysis*): 检查程序的语义正确性, 装饰 AST
4. IR 生成 (*IR Generation*): 基于语法树生成机器无关的中间表示 (如三地址码)

### 1.3.2 中端 (Middle End)

中端在 IR 上执行机器无关的优化 (*Machine-Independent Optimizations*)。优化的优势在于：

- 相比在源码上优化：IR 提供了标准化的形式，更易于处理
- 相比在机器码上优化：IR 提供了机器无关的抽象，并保留了源码信息（如通过符号表获取类型信息）

### 1.3.3 后端 (Back End)

后端负责将 IR 翻译为目标机器代码，并执行机器相关的优化。主要步骤包括：

1. **指令选择 (Instruction Selection)**: 将 IR 映射为目标机器的指令
2. **寄存器分配 (Register Allocation)**: 将变量映射到物理寄存器
3. **指令调度 (Instruction Scheduling)**: 重排指令顺序以利用指令级并行

## 2 编译流程详解

本节深入编译器前端的各个步骤，以具体示例展示编译过程的每一个环节。

### 2.1 词法分析（Lexical Analysis）

核心概念

词法分析 的任务是：根据模式（*Pattern*）找到词素（*Lexeme*），并创建对应的 *Token*。

三个核心概念：

- **Lexeme（词素）**：源代码中的一个字符序列（字符串）
- **Pattern（模式）**：匹配词素的正则表达式规则；如果没有任何模式匹配，则产生词法错误
- **Token（词法单元）**： $\langle \text{token-class}, \text{attribute} \rangle$  对，其中 attribute 用于区分同一类中的不同 token

示例：源码片段 `position = initial + rate * 60` 的词法分析过程

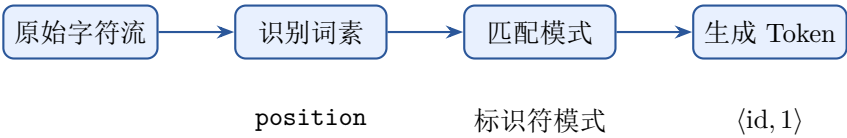


图 4: 词法分析的处理流程

词法分析结果示例：

| 词素（Lexeme） | Token 类 | 属性值      |
|------------|---------|----------|
| position   | id      | 1（符号表入口） |
| =          | =       | —        |
| initial    | id      | 2（符号表入口） |
| +          | +       | —        |
| rate       | id      | 3（符号表入口） |
| *          | *       | —        |
| 60         | number  | 60       |

注意事项

词法分析过程中，如果遇到任何无法匹配任何模式的字符序列，将产生词法错误（*Lexical Error*）。

## 2.2 语法分析（Syntax Analysis）

核心概念

语法分析 的任务是：接收词法分析产生的 Token 流，根据语言的语法规则，构建抽象语法树（*Abstract Syntax Tree, AST*）。

对上述 Token 流构建的 AST：

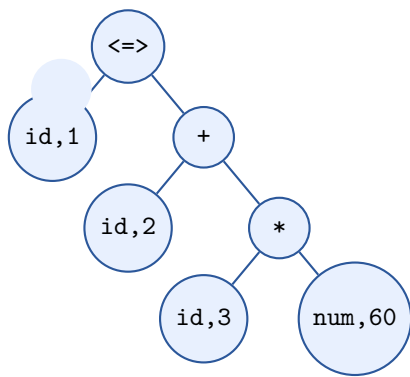


图 5: `position = initial + rate * 60` 的抽象语法树

AST 反映了表达式的运算优先级（乘法优先于加法，赋值优先级最低）。

## 2.3 语义分析（Semantic Analysis）

核心概念

语义分析 检查程序的语义正确性，并为 AST 添加语义装饰（*Semantic Decoration*），如类型信息、作用域信息等。

注意事项

通过语法分析并不意味着程序是正确的！例如以下情况语法上合法但语义错误：

- 类型不匹配（将浮点数赋给期望整数的地方）
- 使用未声明的变量
- 重复声明变量

在语义分析中，编译器会利用符号表（Symbol Table）为 AST 节点标注类型信息：

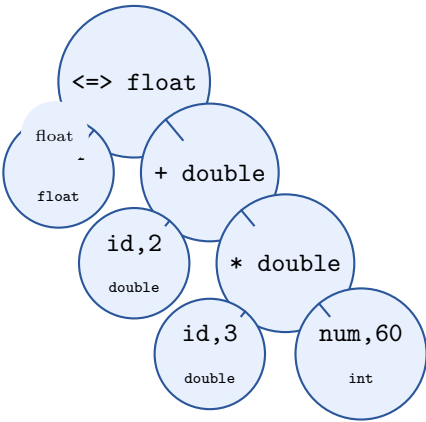


图 6: 添加类型信息后的 AST

语义分析过程中，编译器会进行类型检查：例如 `float = double` 的隐式转换是否合法？若不合法的隐式转换发生，应产生警告或错误。

## 2.4 IR 生成（Intermediate Code Generation）

核心概念

**中间代码生成** 基于 AST 生成机器无关的中间表示（*IR*），最常用的形式之一是三地址码（*Three-Address Code*）。

以图 5 中的 AST 为例，生成的三地址码如下：

Listing 1: 三地址码示例

```
t1 = id3 * 60      // rate * 60
t2 = id2 + t1      // initial + t1
id1 = t2           // position = t2
```

三地址码的特点：

- 每条指令最多包含三个操作数（两个源操作数 + 一个目标操作数）
- 使用临时变量（如 `t1`, `t2`）保存中间结果
- 形式统一，便于后续优化

## 3 中端优化

### 3.1 机器无关优化的特点

#### 核心概念

中端优化在 IR 上执行，是机器无关的 (*Machine-Independent*)。这意味着优化不依赖于特定的硬件架构。

#### 为什么选择在 IR 层做优化？

1. **相对于源代码层面：**IR 提供了标准化的形式，每种语言的前端都生成统一的 IR，因此优化算法可以复用
2. **相对于机器代码层面：**IR 保留了程序的高层信息（如类型信息、控制流结构），这些在机器代码中可能已经丢失
3. IR 抽象掉了与具体机器相关的细节，但保留了足够多的语义信息

### 3.2 优化示例

以前面的三地址码为例，中端可以进行优化：

优化前：

```
t1 = id3 * 60
t2 = id2 + t1
id1 = t2
```

优化后：

```
t1 = id3 * 60
id1 = id2 + t1
```

这里的优化是复写传播 (*Copy Propagation*) + 死代码消除 (*Dead Code Elimination*)：发现 t2 仅仅是将 id2 + t1 的值转发给 id1，因此可以直接将 id2 + t1 赋给 id1，并删除 t2。

## 4 后端：从 IR 到目标代码

后端负责将 IR 转换为目标机器代码，主要包含三个关键步骤。

### 4.1 指令选择 (Instruction Selection)

核心概念

**指令选择** 将 IR 中的每条操作映射为目标机器的具体指令序列，同时进行机器相关的优化。

**示例：**将 IR 指令 `a = a + 1` 翻译为两种不同质量的机器码：  
**朴素翻译 (3 条指令)：**

```
LD  R0, a      // R0 = a
Add R0, R0, #1 // R0 = R0 +
1
ST  a,  R0     // a = R0
```

**优化翻译 (1 条指令)：**

```
INC a          // a = a + 1
```

指令选择的质量直接影响目标代码的大小和执行速度。

### 4.2 寄存器分配 (Register Allocation)

核心概念

**寄存器分配** 的目标是在寄存器数量有限的约束下，尽可能将频繁访问的变量放在寄存器中，因为访问寄存器的速度远快于访问内存。

**核心矛盾：**

- CPU 中寄存器的数量非常有限 (x86-64 仅有 16 个通用寄存器)
- 程序中可能存在成百上千个变量和临时值
- 寄存器分配器必须决定哪些值在何时占用寄存器，哪些值需要「溢出 (Spill)」到内存

## 4.3 指令调度 (Instruction Scheduling)

### 核心概念

**指令调度** 通过重新排列指令的执行顺序，利用目标处理器的指令级并行 (*Instruction-Level Parallelism, ILP*) 能力，以提高代码执行效率。

现代处理器通常具有多条流水线 (Pipeline)，可以在同一时钟周期内执行多条相互独立的指令。指令调度的任务就是重排指令顺序，使彼此独立的指令尽可能并行执行，同时保持程序的语义不变。

## 5 课程结构

### 5.1 内容划分

课程内容按照编译器的三阶段架构进行组织：

| 部分           | 涵盖内容                       |
|--------------|----------------------------|
| Part I 一前端   | 词法分析、语法分析、语义分析             |
| Part II 一中端  | IR 生成、机器无关优化、数据流分析、符号执行    |
| Part III 一后端 | 指令选择、寄存器分配、指令调度、SSA 翻译与反翻译 |
| 扩展主题         | 过程间分析、指针分析                 |

表 2: 课程内容的三部分划分

### 5.2 实验安排

课程配套 8 个实验 (Labs)，与理论内容紧密结合：

| 实验       | 对应内容  |
|----------|-------|
| Lab 1    | 词法分析  |
| Lab 2    | 语法分析  |
| Lab 3    | 语义分析  |
| Labs 4-5 | 中端优化  |
| Lab 6    | IR 生成 |
| Labs 7-8 | 后端    |

表 3: 课程实验安排

### 5.3 课程信息

- 课程主页: <https://qingkaishi.github.io/Compilers.html>
- QQ 群: 1033582067
- 评分方案: 作业 0% + 实验 70% + 期末考试 30%

## 6 第二部分：编译器开发工具

### 6.1 工具概览

编译器开发涉及多种工具，覆盖编译流程的各个阶段。讲义中展示了一个完整的编译器工具链图示。

### 6.2 主要工具分类

#### 1. 词法分析器生成器 (Lexer Generator)

- 代表工具: `lex`, `flex`, ANTLR
- 输入: 基于正则表达式的词法规则描述
- 输出: C/Java 等语言的词法分析器代码

#### 2. 语法分析器生成器 (Parser Generator)

- 代表工具: `yacc`, `bison`, ANTLR
- 输入: 上下文无关文法 (CFG) 描述
- 输出: 语法分析器代码 (LL 分析器或 LR 分析器)

#### 3. 优化框架

- LLVM: 模块化的编译器基础设施, 提供了 IR、优化 Pass、代码生成等功能
- GCC: GNU 编译器套件, 包含完整的编译与优化框架

#### 4. 调试与分析工具

- 语法树可视化工具
- 控制流图 (CFG) / 数据流图 (DFG) 可视化
- 性能剖析 (Profiling) 工具

## 7 第三部分：编译器的应用

### 7.1 高级编程语言的实现

编译器最直接的应用是实现编程语言。此外，编译器技术也是实现领域特定语言（*Domain-Specific Language, DSL*）的关键。DSL 是为特定领域设计的语言（如 SQL 用于数据库查询，Verilog 用于硬件描述），其编译器通常比通用语言编译器更小、更专业化。

### 7.2 计算机体系结构的设计与优化

编译器与硬件设计相互影响。处理器厂商会提供编译器后端以支持其新指令集和微架构特性；同时编译器反馈的信息也会影响下一代处理器的设计决策。

### 7.3 广泛的应用领域

- **人工智能（AI）**：深度学习框架（如 TensorFlow、PyTorch）中的计算图编译与优化（XLA, Glow, TVM 等）
- **游戏开发**：着色器编译器（Shader Compiler），如 HLSL/GLSL 到 GPU 指令的编译
- **嵌入式系统**：针对资源受限设备的交叉编译与优化
- **高性能计算（HPC）**：自动并行化、向量化编译优化

### 7.4 程序翻译（Transpilation）

源语言到源语言的翻译是近年来的研究热点。例如：将 C 代码翻译为 Rust 代码。这类工具需要对源语言和目标语言都有深刻的理解。

### 7.5 软件生产力工具

编译器技术可应用于开发辅助工具：

- 代码格式化工具（如 `clang-format`）
- 代码重构工具（基于 AST 的语义重构）
- 静态分析工具（代码风格检查、潜在 Bug 检测）
- IDE 智能提示（自动补全、跳转定义、查找引用）

## 7.6 安全保证工具

编译器技术在安全领域有广泛应用：

- **Bug 发现：**基于编译器分析的静态漏洞检测（如 Coverity、Infer）
- **程序插桩（Instrumentation）：**在编译时插入额外代码以监控运行时行为（如 AddressSanitizer, ThreadSanitizer）
- **软件防御：**编译时地址空间布局随机化（ASLR）、栈保护（Stack Canary）、控制流完整性（CFI）等

## 8 第四部分：编程语言术语回顾

### 8.1 声明式语言 vs. 命令式语言

核心概念

**命令式语言 (Imperative)** 通过一系列指令 (Commands) 描述如何做 (How to do), 程序显示的是一步步的操作过程。

**声明式语言 (Declarative)** 描述要做什么 (What to do), 而不显式描述执行步骤。逻辑引擎或查询引擎负责确定「怎么做」。

| 命令式语言                | 声明式语言                      |
|----------------------|----------------------------|
| C, C++, Java, Python | Lisp, SQL, Haskell, Prolog |

表 4: 典型的命令式与声明式语言

### 8.2 函数式编程 (Functional Programming)

核心概念

函数式编程中, 程序完全由纯函数 (Pure Functions) 组成, 不允许修改状态或数据 (没有全局变量/可变状态)。

函数式编程的核心特征:

- 1. 没有可变状态——一旦创建, 数据不可变 (Immutable)
- 2. 函数的组合 (Composition) 和应用是主要驱动力
- 3. 高阶函数 (Higher-Order Functions): 函数可以作为参数传递给另一个函数, 也可以作为返回值返回

### 笔记

函数式编程与 SQL 查询的相似性：

```
SELECT SUM(CASE WHEN some_flag = 'X'
                THEN some_column
                ELSE other_column END)
FROM ...
```

这个 SQL 查询没有显式的循环和中间变量，而是声明式地描述了数据处理的方式——这与函数式编程「描述要什么而非怎么得到」的理念一致。

## 8.3 逻辑编程 (Logic Programming)

### 核心概念

逻辑编程的根基在于数学逻辑。程序由事实 (*Facts*) 和规则 (*Rules*) 构成，推理引擎 (Inference Engine) 根据事实和规则自动推导结论。

示例 (使用 Datalog 语法)：

```
// 事实 (Facts)
rainy("Nanjing").
rainy("Beijing").
cold("Beijing").

// 规则 (Rule): 如果一个城市既下雨又冷，则下雪
snowy(c) :- rainy(c), cold(c).

// 查询
.output snowy
```

**推理结果：**北京满足 rainy && cold，因此 snowy("Beijing") 成立；南京只有 rainy 但不满足 cold，因此不输出。

## 8.4 类型系统 (Type Systems)

### 8.4.1 静态类型 vs. 动态类型

核心概念

静态类型 (Static Typing) 和 动态类型 (Dynamic Typing) 的区分在于类型信息获取的时间：

- 静态类型：类型在编译时确定（如 C, Java, Rust）
- 动态类型：类型在运行时确定（如 Python, JavaScript, Ruby）

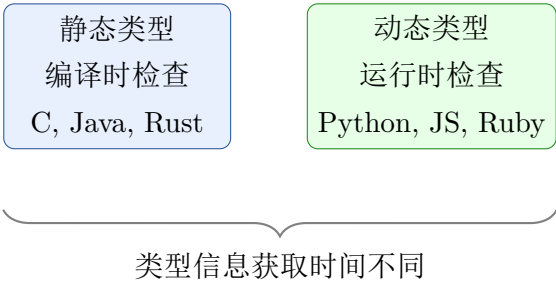


图 7: 静态类型与动态类型的对比

### 8.4.2 强类型 vs. 弱类型

核心概念

强类型 (Strong Typing) 和 弱类型 (Weak Typing) 的区分在于类型区分的严格程度，即语言是否允许隐式类型转换（如从字符串到数字的自动转换）。

| 强类型                | 弱类型                |
|--------------------|--------------------|
| Python, Java, Rust | C, JavaScript, PHP |

表 5: 强类型与弱类型的典型语言

### 注意事项

#### 常见的误解澄清：

- 静态类型  $\neq$  强类型。例如 C 语言是静态类型但弱类型（允许 `void*` 到任意指针的隐式转换）
- 动态类型  $\neq$  弱类型。例如 Python 是动态类型但强类型（不允许字符串和数字直接相加）

## 8.5 作用域（Scoping）

### 核心概念

**静态作用域（Static/Lexical Scoping）** 和 **动态作用域（Dynamic Scoping）** 的区别在于变量引用在何处查找：

- **静态作用域：** 变量引用的绑定在编译时根据程序文本的嵌套结构确定（大多数现代语言采用此方式）
- **动态作用域：** 变量引用的绑定在运行时根据调用栈的上下文确定

## 8.6 函数调用（Function Invocation）

### 核心概念

**值传递（Call by Value）** vs. **引用传递（Call by Reference）** 决定了函数参数在调用时的传递方式。

C/C++ 中的示例对比：

**值传递（Call by Value）：**

```
void swap(int x, int y) {  
    int t;  
    t = x;  
    x = y;  
    y = t;  
}  
// a, b 不变！
```

**引用传递（Call by Reference）：**

```
void swap(int &x, int &y) {  
    int t;  
    t = x;  
    x = y;  
    y = t;  
}  
// a, b 将被交换
```

在值传递中，`x` 和 `y` 是实参的副本；在引用传递中，`x` 和 `y` 是实参的别名（Alias），对它们的修改会直接影响原始变量。

## 8.7 虚函数（Virtual Functions）

### 核心概念

**虚函数** 是在基类中声明、可被派生类重写（*Override*）的成员函数。虚函数是实现运行时多态（*Runtime Polymorphism*）的机制。

- 在 Java 中，任何非 `private`、非 `static`、非 `final` 的方法都是虚函数
- 虚函数的调用在编译时无法确定具体调用哪个实现，需要在运行时通过虚函数表（*vtable*）动态派发（*Dynamic Dispatch*）

示例：

```
void add(List<Integer> list, int y) {  
    list.add(y); // 调 ArrayList.add() 还是 LinkedList.add() ?  
}
```

在编译时，编译器只知道 `list` 的类型是 `List` 接口，无法确定实际运行时对象是 `ArrayList` 还是 `LinkedList`。调用哪个 `add` 方法需要在运行时通过虚函数机制动态决定。

## 9 课程总结

本节作为编译器课程的入门章节，涵盖了以下核心要点：

1. **编译器的定义**：编译器是一种将源语言程序翻译为目标语言程序的软件工具
2. **编译器 vs. 解释器**：编译器产生独立的目标代码；解释器直接执行源语言语句。Java 是典型的三合一方案（编译 + 解释 + JIT）
3. **编译器 vs. 链接器**：编译器负责每个源文件到目标文件的翻译；链接器负责将多个目标文件和库文件组合为最终可执行文件
4. **三阶段架构**：
  - 前端：词法分析 → 语法分析 → 语义分析 → IR 生成
  - 中端：机器无关优化（IR → IR）
  - 后端：指令选择 → 寄存器分配 → 指令调度（IR → 目标代码）
5. **编译器技术的广泛应用**：不仅在编程语言实现中，在 AI、安全、体系结构、游戏等领域都有深入应用
6. **编程语言基础概念**：声明式/命令式、函数式/逻辑式、类型系统、作用域、参数传递、虚函数等

---

本文完

推荐配合原始讲义 Slides 继续深入学习编译器后续章节。